PRODUCT REQUIREMENTS DOCUMENT · CONFIDENTIAL

FBB Persist

Native rebuild — Phase 1

Native iOS and Android. Offline-first. Logger, Movement Library, Nutrition, and Charts. Full migration of workout history, PRs and subscriptions from the current AppRabbit-backed app.

2 platforms	Offline-first	Full migration
Native iOS + Android	**PowerSync over SQLite**	**History, PRs, subs**
Swift 6 / SwiftUI · Kotlin 2 / Compose	Set logging never blocks on network	Apple App Transfer + RevenueCat receipts

Document owners

Engineering lead: Ulhas Mandrawadkar (volunteer rebuild lead) · **Product owner:** Marcus Filly · **Technical owner (FBB):** Satya

How to read this document

Chapters 1–3 frame the problem, scope, and goals — read these together. Chapters 4–7 cover product surfaces (logger, library, nutrition, charts) and are sized so each can be assigned to a single engineer. Chapters 8–11 are the platform layer — auth, sync, video, subscriptions — read by every engineer on the team. Chapter 12 is the migration runbook and must be executed alongside Chapter 8 not after. Chapter 13 lists out-of-scope items — it exists to be referenced when scope creep appears.

Contents

CHAPTER 01

Context and goals

Why this rebuild exists, what it must achieve, and what 'done' means for Phase 1.

1.1 The problem

The current FBB app is a Flutter-based hybrid build delivered by AppRabbit. The programming inside it is the strongest in the category — even one-star reviewers explicitly carve out praise for it. The app shell undermines it. The current iOS rating sits at ~2.5★ across 88 ratings, Android at ~2.7★. The team's own public status page lists data-loss bugs, network errors saving sets, supersets dropping reps, video playback wiping rep state, login fragmentation, and 'network issues leading to lag and errors saving data' as the current top engineering priority. This is not a polish problem — it is an architectural one rooted in a hybrid platform that cannot offer a watch app, cannot ship offline, cannot prevent state loss when the OS reclaims the WebView, and cannot match the logger speed users now expect from Hevy and Strong.

1.2 Goals for Phase 1

- **Match Strong-grade reliability and Hevy-grade logger speed.** Set logging never blocks on network. Video playback never wipes log state. Supersets never drop reps. The user can complete a 90-minute session in airplane mode and have it sync silently when Wi-Fi returns.
- **Reset the App Store narrative.** Ship reliable enough that the next 50 reviews drag the average toward 4.5★ within a quarter of launch.
- **Preserve every existing subscriber and their history.** No subscriber re-purchases. No PR resets. No login confusion. The migration is invisible to *most* users.
- **Support every primitive of FBB's programming language natively.** Tempo (including FBB-specific 'A' for assisted concentric), per-set RPE, supersets, AMRAP, EMOM/E2MOM/E3MOM, For Time, Custom Interval, Olympic complexes, unilateral L/R with separate rest, AMRAP back-offs, percent-of-1RM, Bridge Week deload detection.
- **Lay the foundation for Phases 2–4 without locking it in.** The data model, sync semantics, and content schema must accommodate Apple Watch, Wear OS, Live Activities, AI features, and recovery integration — without scoping any of that work into Phase 1.

1.3 Non-goals for Phase 1

Phase 1 is deliberately conservative on scope. The following are explicitly NOT shipped in Phase 1: Apple Watch app, Wear OS app, Live Activities, Dynamic Island, App Intents/Siri Shortcuts, widgets, on-device voice logging, pose-based form check, AI photo nutrition logging, adaptive macro algorithms, Whoop/Oura integrations, per-track leaderboards, social feed, coach Form Review video upload, multi-language support beyond English, and the v2 admin/coach authoring console. Each is deferred to its proper phase, NOT cancelled.

1.4 What 'done' looks like

Phase 1 exit gate

Phase 1 is done when (1) both apps are live in their stores under the FBB-controlled developer accounts; (2) the existing subscriber base has migrated without a single forced repurchase; (3) the in-workout logger has been used end-to-end (warmup → cooldown) by at least 50 beta users with zero data-loss reports for 14 consecutive days; (4) all four product surfaces — logger, library, nutrition, charts — meet the acceptance criteria in Chapter 14; and (5) the AppRabbit instance has been formally sunset.

CHAPTER 02

In scope, out of scope, non-goals

A precise inventory of what Phase 1 includes — at the feature level, not the product level.

2.1 Phase 1 product surfaces

Surface	What's in	What's NOT in (deferred)
Workout logger	Single-scroll session view; tabular set rows with Prev-set inline; tempo pill; segmented RPE; superset auto-scroll; long-press for skipped/failed; rest timer with auto-start; seven timer modes (Rest/Stopwatch/AMRAP/For Time/Tabata/Custom Interval/EMOM); video pinned as modal with state preservation; Bridge Week deload detection; pre-workout readiness survey.	AI form check; voice logging; Live Activities/Dynamic Island; Apple Watch/Wear OS; HRV-aware autoregulation suggestions.
Movement Library	Browse all demos by muscle / equipment / movement pattern; search; play with HLS adaptive streaming; per-program 'download all' for offline; per-session prefetch overnight on Wi-Fi.	AI semantic search; user-uploaded variations; community-submitted alternates.
Nutrition	Goal-based daily calorie + macro targets (manual user entry, NOT adaptive); meal logging by search / barcode / saved meals; recipe browse; macro ring summary; weekly trend chart.	AI photo logging; adaptive macro algorithm; restaurant chain database depth; meal plan generation.
Progress charts	Body weight; steps (HealthKit / Health Connect); per-exercise weight & e1RM trend; total volume by week; PR feed; user-customisable 'Add Chart' from a fixed schema.	AI insights; comparative charts vs peers; arbitrary cross-source charts.
Account & subs	Sign in with Apple, Google, email magic link; account linking to existing Shopify customer; RevenueCat-managed Persist subscription + workshop entitlements; in-app account deletion.	Family sharing; gift subscriptions; coach-athlete linking.

2.2 Cross-cutting capabilities (every surface)

- Offline-first by default. Every read is from local SQLite; every write commits locally before queueing for upstream sync.
- HealthKit two-way: read steps + body weight; write workouts as HKWorkout (and HealthRecord on Android via Health Connect).

- Push notifications for: rest-timer expiry while backgrounded, scheduled-workout reminders, streak-saving nudges, billing-issue grace warnings.
- Crash reporting (Sentry or equivalent), structured analytics (PostHog or Mixpanel), and a feature-flag layer (PostHog feature flags or LaunchDarkly OSS).
- Accessibility baseline: VoiceOver / TalkBack labels on every interactive element; Dynamic Type up to AX3; Reduce Motion respected; minimum 4.5:1 contrast on body text.

2.3 Explicit non-goals

Things this PRD will NOT pretend are 'small additions'

Apple Watch standalone app (Phase 2). Wear OS standalone app (Phase 2). Live Activities and Dynamic Island (Phase 2). On-device voice set-logging (Phase 4). Pose-based form check (Phase 4). AI photo nutrition logging (Phase 4). Adaptive macro algorithm (Phase 4). Coach Form Review upload + reply (Phase 3). Per-track leaderboards / streaks / cheers (Phase 3). HRV-aware autoregulation (Phase 3). New v2 admin / coach authoring console (parallel workstream — separate PRD). Languages beyond English (Phase 2+).

CHAPTER 03

Brand language and design tokens

Sampled directly from the current FBB app. These are the canonical values for both platforms; do not approximate.

3.1 Color palette

Five core colors plus four neutrals. Sampled from the current FBB app screenshots; the dominant-color histogram of the screenshot confirms these are the actual brand values, not approximations.

FBB Orange	FBB Teal	Orange Tint	Teal Tint
#EE4B0F	#7EBFC7	#FDE9E0	#D1E5E9
Primary action / brand	*Chrome / app shell*	*Callout backgrounds*	*Disabled / hover*

3.1.1 Token table

Token	Hex	iOS Asset Catalog	Android color name	Used for
color.brand.orange	#EE4B0F	FBBOrange	fbb_orange	Primary CTA, selected states, brand
color.brand.orangeDark	#C53D0C	FBBOrangeDark	fbb_orange_dark	Pressed-state on orange CTAs
color.brand.orangeTint	#FDE9E0	FBBOrangeTint	fbb_orange_tint	Callout / banner backgrounds
color.brand.teal	#7EBFC7	FBBTeal	fbb_teal	Header bar, footer, chrome
color.brand.tealDark	#5C9CA4	FBBTealDark	fbb_teal_dark	Pressed-state on teal
color.brand.tealTint	#D1E5E9	FBBTealTint	fbb_teal_tint	Table headers, info state
color.text.primary	#0F172A	InkPrimary	ink_primary	Body text, primary headings
color.text.secondary	#334155	InkSecondary	ink_secondary	Sub-headings, descriptions
color.text.muted	#64748B	InkMuted	ink_muted	Captions, metadata, prev-set

Token	Hex	iOS Asset Catalog	Android color name	Used for
color.surface.background	#F3F4F7	Background	background	Page background between sections
color.surface.card	#FFFFFF	SurfaceCard	surface_card	Card / sheet backgrounds
color.divider	#CBD5E1	Divider	divider	Hairline rules, table cell borders
color.semantic.success	#0F9D58	SemanticSuccess	semantic_success	PR celebration, set complete
color.semantic.warning	#B45309	SemanticWarning	semantic_warning	Billing grace, deload nudge
color.semantic.error	#B91C1C	SemanticError	semantic_error	Failed sets, sync errors

Dark mode (Phase 1 partial)

Phase 1 ships system-default light mode visual; dark mode is wired via the Asset Catalog 'Any Appearance / Dark' variants and Android's `values-night/` resources, but only for system surfaces (text, backgrounds, dividers). Brand orange and teal remain identical across modes. Full dark-mode design polish is Phase 2.

3.2 Typography

System font on both platforms — **`SF Pro`** on iOS, **`Roboto Flex`** on Android. No custom webfont; we ship neither a marketing font nor a 'brand' font. Reasoning: rebuild reliability is the v1 brand value; perfectly-tuned native typography reads as 'a real app' and is faster than any custom-loaded variant.

Token	Size	Weight	Use
type.display	32 pt	Bold (700)	Cover / chapter / large stat values
type.title.1	26 pt	Bold (700)	Section headings (h2 in this PRD)
type.title.2	22 pt	Semibold (600)	Sub-section headings, screen titles
type.title.3	20 pt	Semibold (600)	Card titles, exercise names
type.body	17 pt	Regular (400)	All body text
type.body.bold	17 pt	Semibold (600)	Emphasized body, set values
type.caption	13 pt	Regular (400)	Prev-set, RPE descriptors, metadata
type.mono	15 pt	Regular (400)	Tempo notation, set numbers, RPE values

3.3 Spacing and elevation

Eight-point grid. Spacing tokens: 4, 8, 12, 16, 24, 32, 48, 64. Elevation tokens follow Material 3's 5-level system on Android; iOS uses subtle shadows on cards (radius 8, opacity 8%, y-offset 2).

3.4 Components

- Primary button: orange fill, white text, 44pt min height (iOS HIG / Material accessibility), 12pt corner radius. Pressed state uses orangeDark.
- Secondary button: white fill, ink-primary text, 1pt orange border. Used for destructive-affirmation flows.
- Set row: 56pt min height, 12pt vertical padding. Tap targets on weight/reps/RPE fields are full-row tall, not just text-baseline.
- Card: white fill on bgSoft background, 16pt corner radius, 16pt internal padding.

CHAPTER 04

Workout logger — the heart of the app

The single most-used surface and the place AppRabbit fails worst. Get this right and the App Store narrative resets.

4.1 Architectural invariant

Non-negotiable rule (the AppRabbit bug class fix)

The in-flight workout document is owned by a SINGLE ViewModel rooted at the navigation graph root. Every screen reads and writes through it. No screen owns workout state in its own local @State / remember{} scope. Video and any modal surface are presented as **`fullScreenCover`** on iOS and **`Dialog`** / **`ModalBottomSheet`** on Android — never as a navigation push. AVPlayer is held in a @StateObject so it survives parent re-renders. Violating this rule reproduces the 'playing a video wipes rep scheme' bug class verbatim.

4.2 Session lifecycle

4.2.1 Pre-workout

User taps 'Start Workout' on today's prescribed session. The app:

1. Locks today's prescription as a frozen **`workout_session`** row in local SQLite. Subsequent CMS edits to the program by FBB coaches do NOT affect a session in flight.
2. Shows an optional 30s coach-voice intro (Phase 2 records these for hero sessions; Phase 1 silently skips if absent).
3. Presents a 5-question readiness survey: Sleep, Energy, Soreness, Mood, Stress. Each is a 1–5 chip selector; all skippable. Submitted to a `readiness` table for use in Phase 3 autoregulation.
4. Verifies entitlement once via cached RevenueCat **`CustomerInfo`** and writes **`entitlementVerified=true`** onto the session row. NEVER re-checked mid-workout (see §4.7).

4.2.2 In-workout

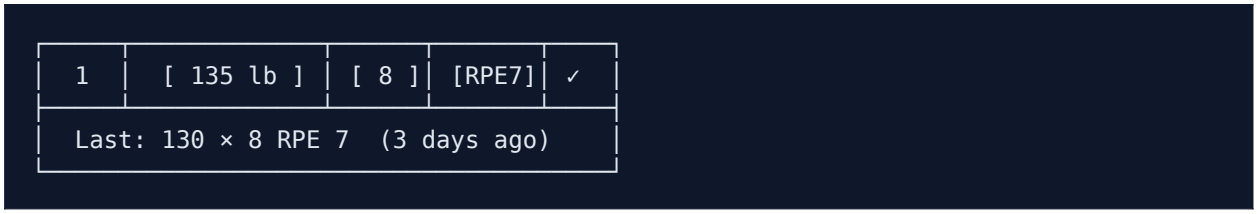
Single-scroll session view. Blocks render top-to-bottom: Warmup, Pre-Fatigue, Strength, Conditioning, Cooldown — current block expanded, others collapsed but tappable. Within each block, exercises render as a single tabular column.

4.2.3 Post-workout

On final-set checkmark or explicit 'Finish Workout', the app: (a) writes a denormalized `workout_summary` row with totals and PR detection results; (b) writes an HKWorkout to HealthKit / HealthRecord to Health Connect tagged with the chosen activity type (default: functionalStrengthTraining); (c) writes the RPE average as iOS 18 Workout Effort Score; (d) shows the post-workout summary screen (totals, PRs, share card).

4.3 Set row layout

Tabular set row, 56pt tall, with five columns: SET # · WEIGHT · REPS · RPE · ✓



- Previous-set values render as a small grey caption *below* the row, NOT as a separate column (preserves horizontal space on 6.1" iPhones and Android compacts).
- Auto-save on field blur. Auto-advance focus to the next incomplete set's weight field within the same exercise.
- Checkmark behavior: marks the set 'completed', strikes it through, fires a `set_completed` event into the outbox, auto-starts the rest timer at the prescribed duration.
- RPE input as segmented stepper buttons: 6 / 6.5 / 7 / 7.5 / 8 / 8.5 / 9 / 9.5 / 10. Default '—'. NOT a slider, NOT freeform numeric.
- Tempo as a monospaced inline pill ⏱️ **`30X1`** under the exercise name with an info-icon tooltip explaining the FBB convention (3s eccentric · 0 hold bottom · X explosive concentric · 1 hold top). Required because the convention is NOT universal — N1 puts pause-after-eccentric in position 2.

4.3.1 Skipped vs failed sets

Long-press the checkmark reveals a 3-option menu: **Complete** / **Failed** / **Skipped**. Failed = red strike + ⚠️ icon, counts as an attempt for PR detection. Skipped = greyed dashed border, doesn't count.

4.4 First-of-class primitives (where no competitor handles FBB's needs)

4.4.1 Olympic complexes

A complex like '1 hang power snatch + 2 hang snatch + 1 OHS' renders as ONE set row per complex execution. The complex descriptor is the exercise label; the unit is *weight × "complex completed"*. Coaches care that the athlete hit '165# × 1 complex' four times, not four individual snatch logs. None of Hevy, Strong, TrainHeroic, Boostcamp, FitNotes, Caliber, Liftin, or Stacked handle this natively.

Optional expansion via a 'Log per movement' toggle on the set row for failed complexes (e.g., athlete hit 1+2 but missed the OHS) — writes additional set rows tagged with the parent complex_id.

4.4.2 Unilateral L/R with separate rest

Two stacked rows per set (L row, R row), each independently checkmarked with its own rest timer. Single-row toggles lose parallel history; two-column rows are cramped on narrow phones. Stacked rows match how lifters actually log unilaterals on paper.

4.4.3 AMRAP back-off prefill

When a coach prescribes '3×5 @ 80%, then 1×AMRAP at 70%', the back-off row prefills as 70% of the just-completed working set's weight (NOT 70% of 1RM — coaches think in terms of the working weight, not the abstract max). Rendered as a greyed placeholder ('140 suggested') tappable to accept and always editable.

4.5 Timer modes

Seven timer modes, adopted verbatim from TrainHeroic's set with bigger tap targets. Each is a distinct render of the in-block area — when a block changes mode, the whole block area swaps, the per-set rows do not.

Mode	When to use	UI shape
Rest	Inter-set rest after a checkmark (auto-starts).	Big circular countdown, ±15s chips, audible+haptic on T-10s and T-0.
Stopwatch	User-driven count-up (warm-ups, mobility flows).	Tap to start / stop / lap. Logs total time.
AMRAP	'AMRAP in 12 min' style finishers.	Big countdown + tap-to-increment 'Round' button. Partial reps logged on stop.
For Time	Capped count-up with rep schedule (21-15-9 etc.).	Count-up clock + checklist of rounds. Time cap → 'Cap+' marker.
Tabata	20s/10s × 8 preset.	Color-coded Work/Rest, 3-2-1 audible, no rep log.
Custom Interval	E2MOM, E3MOM, per-side stretches, etc.	Configurable work + rest, repeat count, optional movement order matrix.
EMOM	Every minute on the minute.	Big minute countdown + minute counter + audible top-of-minute beep. Reps decoupled from timer.

4.6 Video pinning

Tap a movement name to play its demo. Implementation detail (see §4.1 for the architectural reason this is non-negotiable):

4.6.1 iOS

```
// In the SetRow view:
.fullScreenCover(item: $videoMovement) { mvmt in
    DemoVideoView(movement: mvmt, player: workoutVM.player)
        .edgesIgnoringSafeArea(.all)
}

// In WorkoutViewModel (rooted at app shell):
@StateObject private var player = AVPlayer()  // survives child re-renders

// AVPictureInPictureController for 'watch demo while logging':
pipController.canStartPictureInPictureAutomaticallyFromInline = true
```

4.6.2 Android

```
// Compose, demo presented as Dialog rooted at the same NavBackStackEntry:
if (videoMovement != null) {
```

```
    Dialog(onDismissRequest = { videoMovement = null }) {
        DemoVideoView(movement = videoMovement, player = workoutVM.exoPlayer)
    }
}

// In WorkoutViewModel (hoisted to nav-graph root):
val exoPlayer: ExoPlayer = ExoPlayer.Builder(context).build()  // ViewModel-scoped
```

4.7 Subscription expiry mid-workout

Entitlement is checked ONCE at workout start (§4.2.1). If the RevenueCat **`customerInfoUpdateListener`** fires mid-workout indicating revocation, the app DEFERS enforcement until workout completion — show a non-blocking banner ('Your subscription expired — renew before your next workout') but allow the in-flight session to finish and sync. RevenueCat caches CustomerInfo for 3 days when offline; the 14-day grace period at the product level handles billing-issue users gracefully.

4.8 Multi-device single-active-workout

Server enforces single-active-workout per user via a unique partial index on Postgres:

```
CREATE UNIQUE INDEX active_workout
  ON workout_sessions (user_id)
  WHERE status = 'in_progress';
```

If a second device tries to start a workout, the unique-index violation surfaces in PowerSync's **`uploadData()`** callback as a 409. The UI renders: 'You have an active workout on another device. Resume here, or finish there first.' with [Resume] [Take over]. Take-over updates **`last_active_device_id`** and abandons the other session. Far simpler than CRDT-merging two parallel set logs and reflects user reality: one human, one workout at a time.

4.9 Background execution

Both platforms' OS WILL terminate processes during long workouts. The architectural answer: the timer is purely visual. The ground truth is on disk.

- Persist **`started_at`** once at workout start; compute elapsed time as **`Date.now − started_at`** on every render. Never depend on a long-running scheduled timer.
- **Every set checkmark commits to local SQLite synchronously on the user-action thread.** Disk is the ground truth, the rest of the app is a rendering of it.
- iOS rest-timer cues while backgrounded: schedule a **`UNUserNotificationCenter`** local notification at expiry time. The app does not need to be alive. *Do NOT use the silent-audio-keep-alive trick — it gets rejected at App Review.*
- Android: foreground service of type **`FOREGROUND_SERVICE_TYPE_HEALTH`** (introduced in Android 14, no timeout cap unlike dataSync/mediaProcessing). Timer expiry uses **`AlarmManager.setExactAndAllowWhileIdle()`** backed by a high-priority notification.

4.10 Crash-recovery and resume

On launch, if a workout exists with status='in_progress' and last_set_at < 4 hours ago, show a top-of-screen banner: 'Resume your in-progress workout? Started 23 minutes ago • 12 sets logged' with [Resume] [Discard]. After 4 hours of inactivity, server-side auto-marks status='abandoned' (still preserves data; surfaces in history with an 'abandoned' tag). The UI must NEVER block the user from re-entering the app or force a decision — banner-only.

4.11 Logger acceptance criteria

Done = all of:

(1) 50 beta users complete 7 consecutive days of logging with zero data-loss reports. (2) Median tap-to-log latency < 100ms on iPhone 13 / Pixel 6 baseline devices. (3) Airplane-mode 90-min session completes and syncs successfully on Wi-Fi return without manual intervention. (4) Video playback from any movement — including PiP — does not cause any logged set to lose state. (5) All seven timer modes verified against 5 sample FBB sessions each. (6) HealthKit / Health Connect write succeeds 100% on baseline devices (failure-modes documented but not blocking). (7) Bridge Week sessions surface the deload nudge on the correct week.

CHAPTER 05

Movement Library

FBB's catalog of demo videos, browsable and searchable. Phase 1 is delivery; AI semantic search is Phase 4.

5.1 Surface

- Tabs at top: All / Strength / Conditioning / Mobility / Skill.
- Filter chips below: Equipment (Barbell / DB / KB / Bodyweight / Machine / Bands), Muscle group, Movement pattern (Squat / Hinge / Push / Pull / Carry / Locomotion / Rotation).
- Search bar uses SQLite FTS5 (movement_fts virtual table). Indexed fields: name, alternate_names, primary_muscle, equipment, coach_cues.
- Result rows: thumbnail (Sanity-hosted poster), name, primary equipment chip, duration. Tap → modal video view (per §4.6 architecture).

5.2 Data model

```
CREATE TABLE movement (
  id              TEXT PRIMARY KEY,        -- Sanity _id
  name            TEXT NOT NULL,
  alternate_names TEXT,                     -- JSON array
  primary_muscle  TEXT,
  secondary_muscles TEXT,                   -- JSON array
  equipment       TEXT NOT NULL,            -- enum
  movement_pattern TEXT,                    -- enum
  plane           TEXT,                     -- sagittal | frontal | transverse
  joint_action    TEXT,
  unilateral      INTEGER NOT NULL DEFAULT 0,
  difficulty      INTEGER,                  -- 1..5
  coach_cues      TEXT,                     -- markdown
  video_provider  TEXT NOT NULL,            -- 'bunny' | 'mux'
  video_id        TEXT NOT NULL,            -- libraryId/videoGuid OR
muxPlaybackId
  poster_url      TEXT NOT NULL,            -- Sanity CDN
  duration_seconds INTEGER NOT NULL,
  active          INTEGER NOT NULL DEFAULT 1,
  updated_at      INTEGER NOT NULL
);

-- FTS5 virtual table maintained by trigger on movement insert/update
CREATE VIRTUAL TABLE movement_fts USING fts5(
  name, alternate_names, primary_muscle, equipment, coach_cues,
  content='movement', content_rowid='rowid'
);
```

5.3 Substitution graph

Every movement has zero or more **`substitution_rule`** entries. Used by the logger's 'Show alternatives' long-press menu (Phase 1 reads, Phase 3 ranks intelligently).

```
CREATE TABLE substitution_rule (
  id           TEXT PRIMARY KEY,
  movement_id  TEXT NOT NULL REFERENCES movement(id),
  alternate_id TEXT NOT NULL REFERENCES movement(id),
  condition    TEXT NOT NULL,    -- 'no_barbell' | 'shoulder_limit' |
                                 -- 'travel' | 'machine_only' | 'always'
  priority     INTEGER NOT NULL  -- lower = better match
);
```

5.4 Video offline strategy

- **Default: HLS adaptive streaming with a rolling 256 MB cache of last-played demos.** LRU eviction by last-played timestamp; user-managed in Settings → Manage Downloads.
- On program enrollment, opt-in dialog: 'Download all 47 demo videos for PUMP CONDITION 4x (1.2 GB)?'
- Per-session prefetch: the night before each scheduled session, **`BGAppRefreshTask`** (iOS) / **`WorkManager`** periodic work (Android) downloads next session's demos at user-preferred quality if on Wi-Fi and charging.
- Quality picker (Settings): 480p / 720p / 1080p, default 720p. 'Wi-Fi only' default ON.

5.4.1 iOS download lifecycle

```
// AVAssetDownloadConfiguration (iOS 15+, preferred over makeAssetDownloadTask)
let config = AVAssetDownloadConfiguration(
    asset: asset, title: movement.name)
config.primaryContentConfiguration.variantQualifiers = [
    .init(predicate: NSPredicate(format: "variantBitRate <= %d", 1_500_000)),
    .init(predicate: NSPredicate(format: "variantBitRate <= %d", 400_000))
    // CRITICAL: download multiple renditions, never just one,
    // or offline playback fails when no fallback exists (WWDC 2016 #503)
]

let session = AVAssetDownloadURLSession(
    configuration: .background(withIdentifier: "fbb.video.downloads"),
    assetDownloadDelegate: self,
    delegateQueue: .main)

let task = session.aggregateAssetDownloadTask(with: config)
task?.resume()
```

5.4.2 Android download lifecycle

```
// Media3 ExoPlayer DownloadManager
val cache = SimpleCache(
    File(context.filesDir, "video_downloads"),
    NoOpCacheEvictor(),  // never auto-evict downloads
    StandaloneDatabaseProvider(context))
```

```
val downloadManager = DownloadManager(context, databaseProvider, cache,
    DefaultHttpDataSource.Factory(), Executors.newFixedThreadPool(2))

// Restart scheduler (2026-preferred):
downloadManager.requirements = Requirements(NETWORK_UNMETERED + DEVICE_CHARGING)
downloadManager.scheduler = WorkManagerScheduler(context, "fbb-video-downloads")

// Foreground service of type 'dataSync' for download progress notification
```

5.5 Library acceptance criteria

Done = all of:

(1) Full movement catalog renders within 1 second on a cold launch from local SQLite. (2) FTS5 search returns results in <50ms for the test corpus. (3) Per-program 'Download all' completes overnight on Wi-Fi for a representative track without user intervention. (4) An offline session plays every prescribed demo from local cache. (5) Settings → Manage Downloads accurately reports per-program disk usage and supports manual purge.

CHAPTER 06

Nutrition logging

Goal-based macro tracking. Phase 1 is the manual-entry foundation; AI photo logging and adaptive macros are Phase 4.

6.1 Surface

- Daily dashboard: ring summary (Calories outer, Protein middle, Carbs/Fat split inner) + breakdown table + meal cards.
- Add food: search bar (FTS over local food cache) → barcode scanner button → 'Log a recent meal' chip strip → 'Log custom food' fallback.
- Saved meals: user-defined templates (e.g., 'Breakfast burrito', 'Pre-workout shake') for one-tap logging.
- Recipe browse: FBB-curated library (Sanity-managed). Tap a recipe → log a serving → ingredients flow into food_log_entry.

6.2 Food data sources

Source	Role	Coverage	Cost (50K MAU)
USDA FoodData Central	Seeded into the app on first launch	~150 universal items (chicken, rice, eggs, etc.)	$0
Open Food Facts	Primary barcode + free-text search	~4M products, EU-strong	$0
Nutritionix	Paid fallback when OFF misses	~1.9M items, US restaurant strong, NLP parser	~$1,850–3,500/mo

Open Food Facts attribution

OFF is ODbL-licensed: requires attribution and share-alike on derivative DATABASES. Reading-and-displaying records on demand without persisting a derivative DB is the safe path. Include 'Powered by Open Food Facts' attribution in the food-search empty state and About screen.

6.3 Barcode scanning

6.3.1 iOS

```
// AVCaptureMetadataOutput — lowest overhead, 30+ fps live preview
let metadataOutput = AVCaptureMetadataOutput()
metadataOutput.metadataObjectTypes = [
```

```
    .ean13, .ean8, .upce, .upca, .code128
]

// Fallback for still frames using iOS 18 Vision DetectBarcodesRequest
if #available(iOS 18.0, *) {
    let request = DetectBarcodesRequest()
    let observations = try await request.perform(on: image)
}
```

6.3.2 Android

```
// ML Kit BUNDLED variant (model in-app, ~2.4 MB, offline-capable)
// 'com.google.mlkit:barcode-scanning:17.3.0'  — NOT the play-services variant

val options = BarcodeScannerOptions.Builder()
    .setBarcodeFormats(
        Barcode.FORMAT_EAN_13, Barcode.FORMAT_EAN_8,
        Barcode.FORMAT_UPC_A, Barcode.FORMAT_UPC_E,
        Barcode.FORMAT_CODE_128
    ).build()

// CameraX 1.4 + MlKitAnalyzer + Compose (canonical 2026 stack)
```

6.4 Data model

```
CREATE TABLE food (
  id              TEXT PRIMARY KEY,
  source          TEXT NOT NULL,    -- 'usda' | 'off' | 'nutritionix' | 'user'
  source_id       TEXT,              -- fdc_id | barcode | ntx_id
  name            TEXT NOT NULL,
  brand           TEXT,
  serving_size    REAL NOT NULL,
  serving_unit    TEXT NOT NULL,    -- 'g' | 'ml' | 'oz' | 'cup' | 'piece'
  calories        REAL NOT NULL,
  protein_g       REAL NOT NULL,
  carb_g          REAL NOT NULL,
  fat_g           REAL NOT NULL,
  fiber_g         REAL,
  attribution     TEXT,             -- license string for OFF items
  cached_at       INTEGER
);

CREATE TABLE food_log_entry (
  id              TEXT PRIMARY KEY,
  user_id         TEXT NOT NULL,
  consumed_at     INTEGER NOT NULL,
  meal            TEXT NOT NULL,    -- 'breakfast' | 'lunch' | 'dinner' | 'snack'
  food_id         TEXT,              -- nullable if recipe
  recipe_id       TEXT,
  servings        REAL NOT NULL,
  -- denormalized for fast aggregation:
  calories        REAL NOT NULL,
  protein_g       REAL NOT NULL,
```

```
  carb_g          REAL NOT NULL,
  fat_g           REAL NOT NULL,
  fiber_g         REAL,
  source          TEXT NOT NULL,    -- 'manual' | 'barcode' | 'recipe' |
                                     -- 'ai_photo' (Phase 4 reserved)
  photo_id        TEXT,              -- Phase 4 reserved
  created_at      INTEGER NOT NULL,
  updated_at      INTEGER NOT NULL
);
```

6.5 Macro target model (manual, Phase 1)

Phase 1 ships a one-time macro setup wizard at onboarding: user picks goal (Lose / Maintain / Gain), enters body weight + height + activity level + sex, app computes Mifflin-St Jeor TDEE × goal-multiplier and proposes calorie/protein/carb/fat targets. Edits any of the four targets directly. Targets are static until the user changes them. Adaptive algorithms are Phase 4 — the schema reserves room (`target_history` table) but Phase 1 just stores the current target.

6.6 Nutrition acceptance criteria

Done = all of:

(1) Onboarding macro wizard produces a sensible default within 1 minute of first install. (2) Barcode scan → nutrition card visible within 800ms on baseline devices for items in OFF + Nutritionix. (3) Quick-add of a saved meal completes in two taps. (4) Daily macro rings update reactively as items are logged (debounced 250ms). (5) Settings → Nutrition Sources clearly attributes data sources.

CHAPTER 07

Progress charts

Body weight, steps, per-exercise progression, and a user-customisable 'Add Chart' surface.

7.1 Pre-shipped chart templates

- Body weight trend (line, daily resolution, scrollable timeline).
- Daily steps (bar, last 30 days).
- Weekly training volume by week (stacked bar by primary muscle group).
- Big-three 1RMs (line, e1RM via Epley, last 12 weeks per lift).
- Calories vs goal (bar, last 30 days, with goal line overlay).
- PR feed (chronological list with shareable PR cards).

7.2 'Add Chart' schema

Users compose custom charts from a fixed schema. Stored as a row per chart, syncs via PowerSync.

```
CREATE TABLE user_chart (
  id          TEXT PRIMARY KEY,
  user_id     TEXT NOT NULL,
  title       TEXT NOT NULL,
  chart_type  TEXT NOT NULL CHECK(chart_type IN
                ('line','bar','stacked_bar','pie','area')),
  x_source    TEXT NOT NULL,    -- 'date_day' | 'date_week' | 'date_month'
  y_source    TEXT NOT NULL,    -- 'workout_set' | 'body_weight'
                                  -- 'steps' | 'macro'
  y_field     TEXT NOT NULL,    -- 'weight_kg' | 'reps' | 'volume_load'
                                  -- 'protein_g' | etc.
  aggregation TEXT NOT NULL CHECK(aggregation IN
                ('max','min','sum','avg','count','one_rm_epley')),
  filter_json TEXT,             -- JSON: exercise_id, muscle, etc.
  date_range  TEXT NOT NULL DEFAULT 'last_90d',
  created_at  INTEGER NOT NULL,
  updated_at  INTEGER NOT NULL
);
```

7.3 Implementation

- **iOS: Apple Swift Charts (built-in iOS 16+).** LineMark + AreaMark with Catmull-Rom interpolation; chartXSelection for tap-to-detail; chartScrollableAxes(.horizontal) for timelines.
- **Android: Vico 2.2.0+** (com.patrykandpatrick.vico:compose-m3). Active maintenance, KMP-capable, Material 3 theming. *Avoid MPAndroidChart (last release 2019).*

- **Performance budget: ~500 marks per chart before jank.** For 5-year body weight at daily resolution (1,825 points), implement on-device LTTB downsampling — bucket to weekly average when visible domain ≥ 6 months, raw daily when ≤ 30 days. Implemented as SQLite views aggregating by strftime('%Y-%W', ts).

7.4 PR detection

PRs are stored, not derived on read. A Postgres trigger on **`set_logs`** insert maintains the **`prs`** table. Computing on-device means PR rules can't evolve without an app-store review cycle; computing server-side means PR semantics are a deploy.

```
-- e1RM uses Epley: weight * (1 + reps/30) for reps in 1..12
-- Track per (user_id, exercise_id, rep_count_bucket):
--   1RM, 3RM, 5RM, 8RM, 10RM, e1RM-best
--   plus volume PR per (user_id, exercise_id, day)

CREATE TABLE prs (
  user_id          TEXT NOT NULL,
  exercise_id      TEXT NOT NULL,
  pr_kind          TEXT NOT NULL,  -- '1RM' | '3RM' | '5RM' |
                                   -- '8RM' | '10RM' | 'e1RM' | 'volume'
  value            REAL NOT NULL,
  set_log_id       TEXT NOT NULL,
  achieved_at      INTEGER NOT NULL,
  PRIMARY KEY (user_id, exercise_id, pr_kind)
);
```

7.5 Charts acceptance criteria

Done = all of:

(1) All six pre-shipped templates render correctly with seed data. (2) 'Add Chart' wizard produces a working chart from any valid schema combination in under 30 seconds of user effort. (3) 5-year body-weight chart scrolls at 60fps with LTTB downsampling. (4) PR feed shows the user's last 20 PRs with timestamps and source set context. (5) Chart re-renders are debounced 250ms after rapid set logging.

CHAPTER 08

Auth, identity, and account model

Federated sign-in with native ergonomics. Solves the v1 'login twice between web and app' complaint at the architectural layer.

8.1 Provider stack

Provider	Role	Cost (50K MAU)
Supabase Auth	Magic links, email/OTP, JWKS for PowerSync verification	~$25/mo Pro plan
Sign in with Apple	Native iOS via ASAuthorizationController	$0
Google Sign-In	Android via Credential Manager API; iOS via GoogleSignIn-iOS	$0
Postmark (transactional)	Magic-link email delivery + Apple Hide-My-Email relay	~$15–30/mo

Why Supabase over Firebase / Stytch / Clerk / Auth0

Supabase is cheapest at scale (Stytch ~$400/mo, Clerk ~$825/mo, Auth0 $1K-3K/mo at 50K MAU). Asymmetric JWTs work directly with PowerSync's JWKS verification. Open-source escape hatch (self-host Postgres-backed auth if relationship sours). Firebase Auth is also free at this scale but Dynamic Links retired Aug 2025 forces magic links onto Universal Links/App Links + Firebase Hosting — non-trivial migration tax.

8.2 Identity & linking

Three identity providers may attach to one FBB user. Account-linking pattern handles the existing Shopify customer base.

```
CREATE TABLE users (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email        TEXT UNIQUE,            -- canonical, may be Hide-My-Email relay
  display_name TEXT,
  shopify_customer_id TEXT,            -- linked at first auth
  rc_app_user_id TEXT NOT NULL UNIQUE, -- canonical RevenueCat ID
  created_at   TIMESTAMPTZ NOT NULL,
  deleted_at   TIMESTAMPTZ             -- soft-delete; hard at +30d
);

CREATE TABLE identities (
  user_id          UUID REFERENCES users(id),
  provider         TEXT NOT NULL,    -- 'apple' | 'google' | 'email'
  provider_subject TEXT NOT NULL,    -- Apple sub, Google sub, email
```

```
  email           TEXT,
  refresh_token   TEXT,            -- Apple-only, for revocation on delete
  PRIMARY KEY (provider, provider_subject)
);
```

8.3 Token model

- **Access token:** RS256 JWT, 10-minute lifetime, signed asymmetrically. JWKS at **`https://auth.functionalbodybuilding.com/.well-known/jwks.json`**. PowerSync verifies directly without round-tripping to FBB's server.
- **Refresh token:** opaque, 30-day lifetime, single-use rotation with reuse detection (presenting refresh token N after N+1 was issued = treat as breach, invalidate the entire token family).
- **Storage:** iOS Keychain with kSecAttrAccessibleAfterFirstUnlock (allows background refresh); Android EncryptedSharedPreferences backed by Keystore.
- **Claims:** only identity (iss, sub, aud, iat, exp, jti). ***NO entitlements in the JWT.*** Revocation latency would be unacceptable; entitlements live in Postgres and stream to clients via PowerSync.

8.4 Sign in with Apple — three day-1 musts

5. **Token revocation on account deletion.** Apple TN3194 (June 2022) requires calling **`POST appleid.apple.com/auth/revoke`** with the user's stored Apple refresh token at deletion. Store the refresh token server-side at signup. If you forget, prompt re-auth at deletion to obtain a fresh authorization_code.
6. **Hide My Email relay deliverability.** DNS work BEFORE first SiwA user signs up: register transactional domain (e.g., mg.functionalbodybuilding.com) in Apple Developer Console → Sign in with Apple → Email Communication; publish SPF, DKIM (d=functionalbodybuilding.com), Return-Path, DMARC p=quarantine.
7. **Ungroup the SiwA Service ID before Apple App Transfer.** Pre-condition for App Transfer (see Chapter 12). Failure here forces every SiwA user to re-authenticate.

8.5 Account deletion (legal mandate)

Apple Guideline 5.1.1(v) since June 2022; Google Play parity since May 2024. In-app deletion ≤ 3 taps from root.

8. Mark users.deleted_at; schedule 30-day hard-delete worker.
9. Cascade hard-delete: workouts, sets, body_metrics, food_log, charts, messages, media.
10. **Client: `powersync.disconnectAndClear()`**, drop SQLite file, clear URLCache, image caches, Keychain/EncryptedSharedPreferences.
11. Revoke Apple refresh_token via Apple's revoke endpoint.
12. Delete athlete-uploaded form-review videos via Bunny **`DELETE /library/{libraryId}/videos/{videoId}`**.
13. Delete RevenueCat subscriber via **`DELETE /v1/subscribers/{app_user_id}`**.

CHAPTER 09

Offline-first sync architecture

PowerSync over local SQLite, with Sync Streams for content/user/entitlement partitioning. The architectural reason the v1 app's bug list disappears.

9.1 Why PowerSync

Native Swift and Kotlin SDKs (both GA), Postgres source-of-truth, server-authoritative sync, server-driven entitlements. Sync Streams (March 2026 beta) replace legacy Sync Rules and provide on-demand subscription semantics that fit FBB's 'user buys a track mid-session' pattern. Closest production analog: FitWoody (consumer Swift + Supabase + PowerSync). Cost on Pro plan ($49/mo) covers projected ~6 GB synced/month and ~1.8 GB hosted at 50K MAU.

9.2 Sync Streams configuration

```
config:
  edition: 3

streams:
  # Priority 1: in-flight workout — sync first, sync hard
  my_active_workout:
    auto_subscribe: true
    priority: 1
    queries:
      - SELECT * FROM workout_sessions
        WHERE user_id = auth.user_id() AND status = 'in_progress'
      - SELECT * FROM set_logs WHERE session_id IN
          (SELECT id FROM workout_sessions
           WHERE user_id = auth.user_id() AND status = 'in_progress')

  # Priority 2: history + entitled programs (no sub = no sync)
  my_history:
    auto_subscribe: true
    priority: 2
    queries:
      - SELECT * FROM workout_sessions
        WHERE user_id = auth.user_id() AND status != 'in_progress'
      - SELECT * FROM body_metric WHERE user_id = auth.user_id()
      - SELECT * FROM food_log_entry WHERE user_id = auth.user_id()
      - SELECT * FROM prs WHERE user_id = auth.user_id()

  entitled_programs:
    auto_subscribe: true
    priority: 2
    with:
      my_tracks: SELECT track_code FROM entitlements
                 WHERE user_id = auth.user_id() AND active = true
```

```
    queries:
      - SELECT * FROM programs WHERE track_code IN my_tracks
      - SELECT * FROM mesocycles WHERE program_id IN
          (SELECT id FROM programs WHERE track_code IN my_tracks)
      - SELECT * FROM blocks  WHERE mesocycle_id IN ...
      - SELECT * FROM microcycles WHERE block_id IN ...
      - SELECT * FROM days     WHERE microcycle_id IN ...
      - SELECT * FROM prescribed_exercises WHERE day_id IN ...

  # Priority 3: shared, read-only, enormous content
  movement_library:
    auto_subscribe: true
    priority: 3
    query: SELECT * FROM movements WHERE active = true

  mobility_flows:
    auto_subscribe: true
    priority: 3
    query: SELECT * FROM mobility_flows WHERE active = true
```

9.3 Write semantics

- **Client-generated UUID v7** on every row. Idempotent retries are the default; server upserts via **`ON CONFLICT (id) DO UPDATE`**.
- Atomic uploads via **`getCrudTransactions()`**, NOT **`getCrudBatch()`**. A workout session is one atomic unit.
- **Conflict resolution:** last-write-wins per row, deletes-win. No CRDT machinery — single-author workout logs don't need it.
- **Backend write API** MUST apply writes synchronously to Postgres. *No SQS-then-worker pattern in front, or the write checkpoint resolves before data lands and the user sees their just-saved set 'disappear' briefly.*

9.4 Backend connector

Custom Node.js (Fastify) service. Endpoints: **`POST /sync/upload`** (PowerSync calls this with batched writes), **`POST /auth/refresh`**, **`POST /webhooks/sanity`** (content sync), **`POST /webhooks/revenuecat`** (entitlement sync). Hosted on Fly.io or Railway, multi-region. Postgres primary + read replica.

9.5 Known footguns to ticket day-1

(1) PowerSync DBs cannot be opened from iOS App Extensions (Widgets, Live Activities). Workaround: dump a Codable JSON snapshot to App Group on every meaningful update. Phase 2 constraint, not Phase 1. (2) PowerSync GRDB / Room ORM bridges are alpha as of May 2026 — use the standard PowerSync.execute / .watch path, NOT the bridges. (3) Writes resolve faster than the database in some configurations — must be synchronous to Postgres.

9.6 Local DB choice

- **iOS: GRDB 7.10+** for typed query support and SwiftUI ValueObservation, used alongside (not under) PowerSync. PowerSync owns the SQLite instance; GRDB provides query ergonomics on top of the same DB.
- **Android: Room 2.8** with Flow-based DAOs. Same pattern — PowerSync owns the DB, Room DAOs query it.
- **Schemas duplicated between platforms (~15-25 tables; the duplication cost is small).** KMP/SQLDelight is deferred to a v1.5 carve-out for the workout state machine + e1RM math.
- **Encryption:** iOS FileProtection (NSFileProtectionComplete) + Android default disk encryption. *Skip SQLCipher in Phase 1 — no HIPAA scope, 2-3.5× write cost not justified.*

CHAPTER 10

Video infrastructure

Bunny Stream for delivery, Mux Data SDKs for analytics. ~5× cheaper than Mux Video at FBB's scale, with no analytics gap.

10.1 Provider stack

Layer	Provider	Why	Cost (50K MAU)
Encoding + storage	Bunny Stream	Free encoding; ~$0.01/GB storage	~$2/mo
Delivery (HLS)	Bunny Volume Network	$0.005/GB; native player consumes HLS directly	~$1,250/mo
Player	AVPlayer (iOS) / Media3 ExoPlayer (Android)	Native, no SDK dependency	$0
QoS analytics	Mux Data SDKs	Free at FBB's view volume; 1-line attach to any HLS source	$0
Posters / thumbnails	Sanity CDN	Already used for content; no separate vendor	$0

Bunny vs Mux Video at scale

Mux Video costs ~$5,900/mo at 50K MAU vs Bunny's ~$1,250/mo — a $56K/year saving. The gap widens linearly with growth. Mux Data SDKs work with any HLS source including Bunny, so we get Mux's best-in-class QoS dashboard without the Mux Video price tag. Custom dimensions (10 free 'customN' slots) make programId, mesocycleId, dayId, movementId, userTier first-class filters in the dashboard.

10.2 Polyglot video reference (Sanity)

Reference object at the schema level so a movement can switch providers without a content migration. Phase 1 ships Bunny-only; Mux is reserved for live workout streaming in Phase 2 if it materialises.

```
// Sanity schema — externalVideo object
{
  provider: 'bunny' | 'mux',
  // bunny:
  bunnyLibraryId: string,
  bunnyVideoGuid: string,
```

```
  // mux:
  muxPlaybackId: string,
  // shared:
  durationSeconds: number,
  aspectRatio: '16:9' | '9:16' | '1:1',
  posterImage: image  // Sanity-hosted
}

// Client builds playback URL at runtime — never embed in Sanity
const url = video.provider === 'bunny'
  ? `https://vz-${BUNNY_CDN}.b-cdn.net/${video.bunnyVideoGuid}/playlist.m3u8`
  : `https://stream.mux.com/${video.muxPlaybackId}.m3u8`
```

CHAPTER 11

Subscriptions

RevenueCat as the entitlement source of truth across iOS, Android, and Stripe.

11.1 SDKs and pricing

- **`purchases-ios`** v5.x (StoreKit 2 default, iOS 13+, Swift 6 toolchain).
- **`purchases-android`** v8/v9.x (BillingClient 7+, minSdk 21, first-class Compose).
- Pricing at FBB's ~$2M ARR ≈ $166K MTR. Free tier ends at $2,500 MTR. Pro plan at 1% of MTR = **~$1,667/month (~$20K/year)** — within build-vs-buy band.

11.2 Entitlement model — multi-attach

One `persist` entitlement granted by Persist subscription products. Plus one entitlement per workshop (`workshop_pump40`, `workshop_aerobic40`, etc.) where each workshop entitlement has BOTH the workshop's standalone product AND the Persist subscription products attached. This encodes 'Persist unlocks all current and future workshops' while allowing à-la-carte workshop purchases to remain lifetime after Persist cancellation. Single-entitlement models can't represent that.

11.3 Webhook architecture (the glue to PowerSync)

```
// POST /webhooks/revenuecat
// 1. Verify Authorization-header bearer secret
// 2. Write raw payload to webhook_events inbox (UNIQUE on event.id)
// 3. Return 200 within 60s
// 4. Background worker derives entitlement upsert:
//
//    On INITIAL_PURCHASE / RENEWAL / PRODUCT_CHANGE / UNCANCELLATION:
//      INSERT entitlements (user_id, track_code, active, expires_at)
//      ON CONFLICT (user_id, track_code) DO UPDATE
//        SET active=true, expires_at=excluded.expires_at
//
//    On CANCELLATION / EXPIRATION / SUBSCRIPTION_PAUSED:
//      UPDATE entitlements SET active=false WHERE ...
//
// PowerSync streams entitlement table → device within seconds.
// Daily reconciliation: GET /v1/subscribers/{app_user_id}
// to backfill any missed events.
```

11.4 Migration runbook (T-60 to T+30)

Full runbook is Chapter 12. Subscription-specific milestones:

- **`T−60`**: Configure App Store Server Notifications v2 → RC URL on prod AND sandbox; same for Google RTDN via Pub/Sub; install RevenueCat App from Stripe Marketplace.

- **`T−30`**: Server-side import of historical receipts via **`POST /v1/receipts`** (Apple full receipts; Google tokens <60 days old).
- **`T+0`**: Submit native app under same Bundle ID; force-update banner on old AppRabbit app.

CHAPTER 12

Migration from AppRabbit

Identified as the single highest-risk Phase 1 workstream. Begins parallel to engineering, NOT after.

12.1 Risk register

Data class	Risk	Mitigation
Apple App Transfer (Cliff Kohut → FBB)	🔴 High	Initiate week 1. Without it, every existing iOS subscriber must re-purchase. Pre-conditions: ungroup SiwA Service ID, generate app-specific shared secret, ensure no IAP product ID conflicts in recipient account.
Google Play app transfer	🔴 High	Parallel to Apple. Initiate week 1.
End-user workout history / set logs / PRs	🔴 High	AppRabbit ToS silent on portability; no documented API. Plan manual export workstream — browser-driven scraping of admin dashboard. Pair with pre-cutover prompt asking each user to confirm/correct their PRs (turns risk into re-engagement).
Form-review videos / chat archives	🔴 High	Best-effort export; document data loss if unavailable. Communicate transparently to users.
Movement Library + demo videos	🟡 Medium	Owned by FBB but on AppRabbit infra; bulk download requires AppRabbit cooperation.
Stripe web subscriptions	🟡 Medium	Portable directly via Stripe IF FBB is merchant of record — verify.
Program content (workouts authored by FBB)	🟢 Low	FBB likely has master copies in their authoring workflow regardless. Re-authored into Sanity directly.

Action item, week 1

Pull AppRabbit's executed master agreement. Identify the Apple Account Holder (Cliff Kohut?) and confirm willingness to initiate App Transfer. These two actions gate every other Phase 1 decision. Without them, the rebuild ships under a new Bundle ID and every subscriber re-purchases.

12.2 Migration runbook

12.2.1 T-60 days — foundation

14. Sanity project provisioned; schemas merged.

15. Postgres primary + read replica provisioned; migrations applied.
16. PowerSync project + Sync Streams configured (§9.2).
17. RevenueCat project; iOS .p8 + App-Specific Shared Secret + ASC API Key uploaded.
18. App Store Server Notifications v2 → RC URL on prod AND sandbox.
19. Google Real-Time Developer Notifications via Pub/Sub → RC.
20. Stripe Marketplace → install RevenueCat App.
21. RC dashboard: enable *'Track new purchases from server-to-server notifications'* — this captures renewals on the OLD app version BEFORE any user installs the new app.

12.2.2 T-45 days — App Transfer

22. Cliff Kohut (current Apple Account Holder) initiates App Transfer to FBB's new ASC team.
23. Pre-conditions verified: SiwA Service ID ungrouped; app-specific shared secret generated; no IAP product ID conflicts in recipient account.
24. Equivalent Google Play app transfer initiated.
25. Plan that users will be forced to re-login ONCE after the post-transfer update (keychain migration breaks).

12.2.3 T-30 days — historical backfill

26. Server-side import: POST full base64-encoded Apple receipts and Google purchase tokens to **`POST /v1/receipts`** with the canonical FBB user_id.
27. Apple constraint: must be the FULL receipt, not **`latest_receipt_info`**.
28. Google constraint: tokens expired >60 days cannot be imported.
29. Bulk imports do NOT trigger webhooks — use REST API rather than CSV.
30. Workout history scrape: browser-automation extraction from AppRabbit admin dashboard, normalized into the Phase 1 Postgres schema.

12.2.4 T-14 days — beta

31. Ship via TestFlight / Play internal track.
32. Verify existing entitlement carries over without 'Restore Purchases' tap. RC SDK auto-detects on-device transactions on first launch.
33. Beta cohort = 50 highly engaged Persist subscribers, hand-picked, plus FBB internal team.
34. 14-day beta gating exit gate: zero data-loss reports, zero failed entitlement carryovers.

12.2.5 T+0 — cutover

35. New native app submitted under same Bundle ID / package name.
36. Force-update banner on old AppRabbit app: 'FBB is now native. Update to continue training.'
37. Communicate to users 7, 3, and 1 day in advance via email + in-app banner.

12.2.6 T+30 — sunset

38. Daily reconciliation: compare RC subscriber count vs. AppRabbit's last known vs. Stripe's active count. Any delta >1% triggers investigation.
39. Sunset old AppRabbit version (remove from stores; data retention per legal).

40. Final form-review video / chat archive export (best-effort).

CHAPTER 13

Out of scope for Phase 1

This chapter exists for one reason: to be referenced when scope creep appears.

If a feature is below, and someone asks 'can we just add this to Phase 1', the default answer is no. Each is deferred to a specific phase with intent — not cancelled, not forgotten.

Feature	Phase	Reason for deferral
Apple Watch standalone app	Phase 2	Requires WorkoutKit + ActivityKit integration with workout state machine. Best built once core logger is stable.
Wear OS standalone app	Phase 2	Compose for Wear OS path; same dependency on stable core.
Live Activities + Dynamic Island	Phase 2	PowerSync DBs cannot open from iOS App Extensions; needs the App Group snapshot pattern designed.
App Intents + Siri Shortcuts	Phase 2	Same App Extension constraint.
Home Screen widgets	Phase 2	Same App Extension constraint.
Per-track leaderboards / streaks	Phase 3	Requires social graph + identity moderation flows. Distinct workstream.
Coach Form Review video upload + reply	Phase 3	Requires moderation pipeline + coach-side admin tools.
RPE-aware progression suggestions	Phase 3	Requires e1RM history baseline + readiness-survey baseline.
Equipment-aware Smart Replace algorithm	Phase 3	Requires substitution graph fully populated + ranking model.
Whoop / Oura / HealthKit HRV autoregulation	Phase 3	Requires multi-source recovery model + opinionated nudge engine.
On-device voice set-logging (WhisperKit)	Phase 4	On-device LLM scope distinct from core sync.
Pose-based form check	Phase 4	MediaPipe Pose Landmarker / Vision integration. Liability framing required.
AI photo nutrition logging	Phase 4	Vision/LLM scope. Schema reserved in §6.4.
Adaptive macro algorithm	Phase 4	Needs nutrition baseline first. Schema reserved in §6.5.
LLM next-workout briefer	Phase 4	Needs workout history baseline first.

Feature	Phase	Reason for deferral
Multi-language support	Phase 2+	Sanity i18n schema is Phase-1-ready; UI strings localized when languages arrive.
v2 Coach / Admin authoring console	Parallel	Separate workstream. Sanity Studio is the Phase 1 surface; the v2 console is the Phase 3 polish.

CHAPTER 14

Acceptance criteria and exit gates

What 'Phase 1 done' looks like, measured at the seam where engineering hands off to launch.

14.1 Functional gates

Surface	Gate	Verified by
Logger	50 beta users complete 7 consecutive days of logging with zero data-loss reports.	Beta telemetry
Logger	Median tap-to-log latency < 100ms on iPhone 13 / Pixel 6 baseline.	Internal benchmark
Logger	Airplane-mode 90-min session completes and syncs successfully on Wi-Fi return without manual intervention.	QA scenario
Logger	Video playback (including PiP) does not cause any logged set to lose state.	QA scenario × 5 movements
Logger	All seven timer modes verified against 5 sample FBB sessions each.	Coach review
Logger	Bridge Week sessions surface the deload nudge on the correct week.	Test program rollout
Library	Full movement catalog renders in <1s on cold launch from local SQLite.	Internal benchmark
Library	FTS5 search returns results in <50ms for the test corpus.	Internal benchmark
Library	Per-program 'Download all' completes overnight on Wi-Fi without user intervention.	Beta scenario
Nutrition	Onboarding macro wizard produces a default in <1 minute of first install.	QA timed run
Nutrition	Barcode scan → nutrition card visible within 800ms on baseline devices for OFF + Nutritionix items.	QA × 50 items
Charts	All six pre-shipped templates render correctly with seed data.	QA
Charts	5-year body-weight chart scrolls at 60fps with LTTB downsampling.	Profiler
Auth	Existing Persist subscriber signs in via Apple/Google/email and lands on dashboard with full history visible.	Beta scenario × 10 users
Migration	100% of in-scope existing iOS subscribers retain entitlement without Restore Purchases.	RC reconciliation

Surface	Gate	Verified by
Migration	100% of in-scope existing Android subscribers retain entitlement.	RC reconciliation

14.2 Non-functional gates

- **Crash-free sessions ≥ 99.5%** (measured by Sentry over the 14-day beta).
- **App Store + Play Store privacy declarations complete and accurate.** PrivacyInfo.xcprivacy shipped on iOS.
- **In-app account deletion verified end-to-end:** PowerSync disconnectAndClear, SiwA token revocation, RC subscriber delete, Bunny media delete, Postgres soft-delete with 30-day hard-delete worker.
- **Accessibility:** VoiceOver / TalkBack labels on every interactive element. Dynamic Type AX1 verified. Reduce Motion respected.
- **DPIA completed before EEA launch.** Privacy policy URL hosted and linked in-app.

14.3 Launch gate

Phase 1 launches when:

Every functional gate above is met. Crash-free sessions stay ≥ 99.5% for 14 consecutive days. Migration reconciliation reports < 1% delta on subscriber counts. The FBB internal team has trained for one full week using the new app exclusively. Marcus has personally completed his own programmed sessions on both platforms. The old AppRabbit app's force-update banner is staged but not yet enabled.

End of PRD · Phase 1 · Functional Bodybuilding native rebuild